



Desarrollo de Software para Móviles – DSM941

Ciclo 01 - 2026

Inge. Alexander Alberto Siguenza Campos

Foro de Discusión 2 –

**Desarrollo de Aplicación de Control de Gastos Personales con Firebase y Kotlin en
Android Studio**

Integrantes:

Alumnos	Carne
Katya Maria Hernández Pérez	HP221350
Juan Carlos Ramírez Chávez	RC231487
Melvin Eduardo Robles Rodas	RR191220
David Roberto Ferrer Coto	FC243112

Índice

Introducción	3
Investigación sobre autenticación en Firebase	4
¿Qué es Firebase Authentication?	4
Tipos de autenticación disponibles	4
Autenticación con correo electrónico y contraseña	4
Autenticación con Google (OAuth 2.0)	5
Autenticación por número de teléfono	5
Autenticación con proveedores federados (Facebook, Apple, X, GitHub, Microsoft)	5
Autenticación anónima	6
Autenticación mediante enlace de correo (Email Link / Passwordless)	6
Ventajas generales de Firebase Authentication	7
Limitaciones y consideraciones	7
Flujo técnico de autenticación con Firebase	8
Capturas del funcionamiento de la Aplicación	9
Conclusiones	14
Conclusión general	15

Introducción

El crecimiento de aplicaciones móviles ha sido excepcional en la última década. El sistema Android, gracias a su extensa base de usuarios en todo el mundo, se ha establecido como uno de los ecosistemas más importantes para desarrollar software destinado a dispositivos móviles. En este escenario, el lenguaje de programación Kotlin ha surgido como la opción predilecta por Google para el desarrollo nativo de aplicaciones Android, gracias a su sintaxis breve, su interoperabilidad con Java y su resistencia ante errores habituales como las excepciones de puntero nulo.

Este trabajo pertenece a la Unidad 4 de la materia, cuyo propósito esencial es la creación de una aplicación móvil operativa que facilite a los usuarios el control de sus gastos personales. La propuesta combina dos servicios esenciales de la plataforma Firebase de Google: Firebase Authentication, para la gestión segura de las identidades de los usuarios, y Cloud Firestore, para el almacenamiento y la sincronización de datos en la nube. Ambas tecnologías son recursos empresariales disponibles para grupos de desarrollo de cualquier magnitud.

La aplicación creada permite a los usuarios iniciar sesión a través de correo electrónico o cuenta de Google, anotar sus gastos con datos como nombre, cantidad, categoría y fecha, revisar un registro de transacciones y observar el total acumulado de gastos por mes. Este conjunto de características responde a una necesidad diaria tangible: la gestión económica personal, utilizando conocimientos técnicos específicos sobre autenticación, almacenamiento de datos y diseño de interfaces en Android Studio.

El presente documento está organizado en cuatro secciones principales. En primer lugar, se presenta una investigación teórica sobre los mecanismos de autenticación disponibles en Firebase, sus ventajas, limitaciones y casos de uso. A continuación, se documenta el proceso de implementación técnica paso a paso. Posteriormente, se muestran evidencias del funcionamiento de la aplicación mediante capturas de pantalla. Finalmente, el equipo presenta sus conclusiones individuales sobre el proceso de aprendizaje.

Investigación sobre autenticación en Firebase

¿Qué es Firebase Authentication?

Firebase Authentication es un servicio de backend proporcionado por Google que simplifica la integración de sistemas de autenticación en aplicaciones móviles y web. Su objetivo es reducir la dificultad relacionada con la gestión de credenciales de usuario: resguardo seguro de contraseñas, tokens de sesión, recuperación de cuentas y protección contra ataques de fuerza bruta, entre otros. En vez de crear estos mecanismos desde el principio, el desarrollador puede incorporar Firebase Authentication a través de un SDK y dejar la administración de identidades en la infraestructura de Google.

El servicio soporta varios proveedores de identidad, lo que facilita a los usuarios seleccionar su forma de acceso preferida. Internamente, Firebase maneja los tokens de autenticación empleando el estándar JSON Web Token (JWT), lo que asegura la compatibilidad con otros servicios y APIs. Cada usuario autenticado obtiene un identificador único (UID) que se puede utilizar para vincular datos en Firestore o en otras bases de datos.

Tipos de autenticación disponibles

Firebase Authentication ofrece una variedad de métodos de autenticación que se adaptan a distintos contextos y necesidades. A continuación, se describen los principales tipos:

Autenticación con correo electrónico y contraseña

Es el método más tradicional y ampliamente utilizado. El usuario proporciona una dirección de correo y una contraseña; Firebase verifica las credenciales y emite un token JWT. Las contraseñas se almacenan de forma cifrada usando el algoritmo bcrypt, lo que garantiza que ni siquiera los administradores del proyecto pueden acceder a ellas en texto plano. Este método es sencillo de implementar y no requiere que el usuario posea una cuenta en ninguna red social o servicio externo.

Entre sus beneficios resalta su adaptabilidad, puesto que cualquier individuo con una dirección de correo puede inscribirse. Sus principales restricciones son la necesidad de un proceso para recuperar la contraseña y la obligación del usuario de resguardar una contraseña segura. Es particularmente apropiado para aplicaciones internas, plataformas educativas y sistemas que necesitan un control minucioso del acceso

Autenticación con Google (OAuth 2.0)

La autenticación con Google utiliza el protocolo OAuth 2.0 para delegar la verificación de identidad a los servidores de Google. El usuario es redirigido a una pantalla de consentimiento de Google, donde autoriza el acceso a su información básica de perfil. Una vez concedido el permiso, Firebase recibe un token de identidad de Google y crea o asocia una cuenta de usuario en el proyecto.

Las ventajas de este método incluyen la eliminación de la fricción del registro, ya que el usuario no necesita crear ni recordar una contraseña adicional. Además, aprovecha la infraestructura de seguridad de Google, incluyendo la verificación en dos pasos si el usuario la tiene activada. La principal limitación es la dependencia de una cuenta de Google activa y de conectividad a Internet. Es ideal para aplicaciones de consumo, herramientas de productividad y cualquier producto orientado al público general.

Autenticación por número de teléfono

Este método envía un código de verificación de un solo uso (OTP, One-Time Password) al número de teléfono del usuario mediante un mensaje SMS. El usuario ingresa el código recibido para completar la autenticación. Firebase gestiona el envío del SMS a través de su infraestructura, aunque en producción se incurre en costos variables según el país de destino.

Su principal ventaja es la accesibilidad: prácticamente todos los usuarios móviles tienen un número de teléfono, independientemente de si poseen una cuenta en algún servicio en línea. Es especialmente útil para aplicaciones de servicios financieros, aplicaciones de delivery y plataformas de salud donde la verificación de identidad real es prioritaria. Sus limitaciones incluyen el costo de los SMS, la dependencia de señal de red y la posibilidad de intercambio de SIM (SIM swapping) como vector de ataque.

Autenticación con proveedores federados (Facebook, Apple, X, GitHub, Microsoft)

Firebase permite la integración con múltiples proveedores de identidad mediante el protocolo OAuth 2.0 o SAML. Entre los proveedores soportados se encuentran Facebook, Sign in with Apple, Twitter/X, GitHub y Microsoft. Cada uno requiere la creación de una

aplicación en el portal del proveedor correspondiente y la configuración de las credenciales en Firebase Console.

El principal beneficio de estos métodos es la reducción del abandono en el proceso de registro, ya que los usuarios pueden utilizar cuentas que ya tienen activas. Sign in with Apple es obligatorio para aplicaciones iOS que ofrezcan otras opciones de inicio de sesión federado, según las políticas de la App Store. La complejidad de configuración y las posibles restricciones de los proveedores representan las principales limitaciones.

Autenticación anónima

Firebase permite crear sesiones anónimas para usuarios que aún no desean registrarse. Se genera una cuenta temporal con un UID único que permite acceder a recursos de Firestore o Realtime Database con las reglas de seguridad configuradas. Posteriormente, esta cuenta anónima puede vincularse a un método de autenticación permanente sin perder los datos asociados.

Este método es ideal para aplicaciones que ofrecen una experiencia previa al registro, como juegos móviles, aplicaciones de e-commerce con carrito de compras antes del checkout, o cualquier producto donde reducir la fricción inicial sea una prioridad de negocio. Su limitación más relevante es que las cuentas anónimas se eliminan si el usuario desinstala la aplicación sin haber vinculado un método permanente.

Autenticación mediante enlace de correo (Email Link / Passwordless)

En lugar de requerir una contraseña, Firebase envía un enlace de autenticación al correo electrónico del usuario. Al hacer clic en el enlace desde el mismo dispositivo, la sesión se abre automáticamente. Este enfoque, conocido como autenticación sin contraseña (passwordless), elimina el riesgo de contraseñas débiles o reutilizadas, ya que cada enlace es de un solo uso y tiene una validez limitada.

Es especialmente útil para aplicaciones donde la seguridad es crítica y se desea reducir la superficie de ataque relacionada con contraseñas. La principal limitación es que el usuario debe tener acceso a su correo electrónico en el momento de la autenticación, lo que puede representar una barrera en entornos con conectividad limitada.

Ventajas generales de Firebase Authentication

La adopción de Firebase Authentication ofrece beneficios concretos para equipos de desarrollo de cualquier tamaño. En primer lugar, reduce drásticamente el tiempo de implementación: lo que podría requerir semanas de desarrollo para un sistema de autenticación propio se logra en horas con Firebase. En segundo lugar, el servicio gestiona automáticamente aspectos críticos de seguridad como el almacenamiento cifrado de contraseñas, la rotación de tokens y la protección contra ataques de enumeración de cuentas.

Adicionalmente, Firebase Authentication se integra de manera nativa con el resto del ecosistema Firebase, en particular con Cloud Firestore y Realtime Database, mediante el sistema de reglas de seguridad. Esto permite definir permisos a nivel de documento basados en el UID del usuario autenticado, garantizando que cada usuario solo pueda acceder a sus propios datos. La escalabilidad del servicio es otro punto a favor: Firebase Authentication puede gestionar desde unos pocos usuarios hasta millones sin que el equipo de desarrollo deba preocuparse por la infraestructura subyacente.

Limitaciones y consideraciones

A pesar de sus beneficios, Firebase Authentication tiene restricciones que deben tomarse en cuenta al planificar la estructura de una aplicación. La dependencia de los servidores de Google significa que, en caso de una interrupción del servicio, los usuarios no podrán iniciar sesión. A pesar de que Google ofrece elevados niveles de disponibilidad (SLA de 99.95%), esta dependencia podría ser inaceptable en determinados contextos de regulación o seguridad nacional.

Otra limitación significativa es la adaptación del proceso de autenticación: si bien Firebase ofrece interfaces de usuario prediseñadas (FirebaseUI), estas podrían no cumplir con las necesidades de marca o la experiencia del usuario en ciertos proyectos. En tales situaciones, el equipo tiene que desarrollar su propia interfaz utilizando directamente el SDK. Finalmente, la verificación telefónica implica costos fluctuantes que pueden aumentar considerablemente en aplicaciones con un gran número de usuarios en naciones con altas tarifas de SMS.

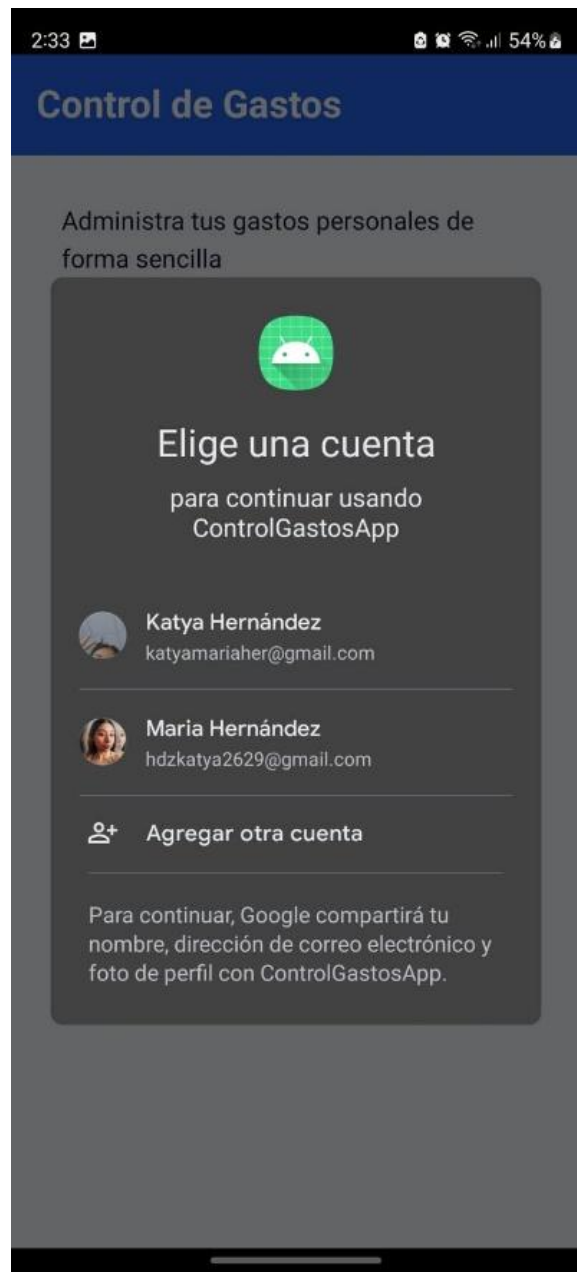
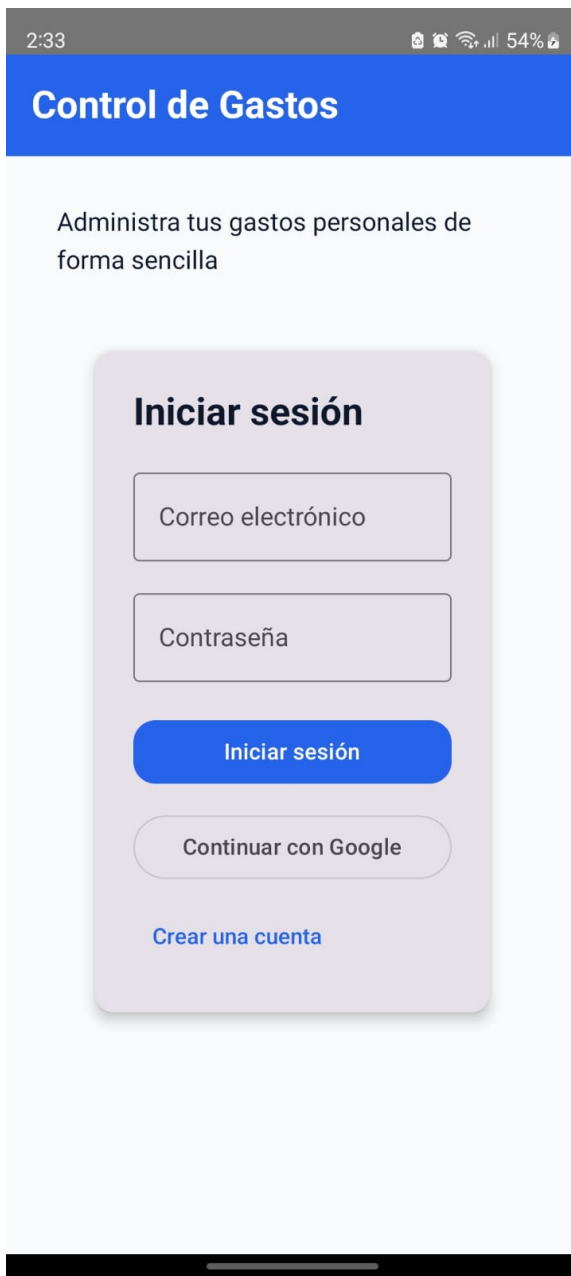
Flujo técnico de autenticación con Firebase

El proceso de autenticación en Firebase sigue un flujo bien definido que garantiza la seguridad de las credenciales y la correcta gestión de sesiones. Independientemente del método utilizado, el flujo general comprende las siguientes etapas: (1) el cliente envía las credenciales al SDK de Firebase; (2) el SDK comunica las credenciales al servidor de Firebase Authentication mediante HTTPS; (3) Firebase verifica las credenciales con el proveedor de identidad correspondiente; (4) si la verificación es exitosa, Firebase emite un token de identificación JWT firmado con una clave privada de Google; (5) el cliente almacena el token de forma segura y lo incluye en cada solicitud a los servicios de Firebase; y (6) los servicios como Firestore validan el token antes de conceder acceso a los datos (Google Identity, 2023).

Este flujo garantiza que las credenciales del usuario nunca sean procesadas directamente por el código de la aplicación, sino que sean gestionadas por la infraestructura segura de Google. El token JWT tiene una validez de una hora y Firebase lo renueva automáticamente en el cliente cuando está próximo a expirar, lo que garantiza una experiencia de usuario fluida sin requerir re-autenticaciones frecuentes.

Capturas del funcionamiento de la Aplicación

Pantalla de autenticación de la aplicación. Se visualiza el formulario de inicio de sesión con campos para correo electrónico y contraseña, el botón de autenticación mediante Firebase Authentication con correo/contraseña, y la opción de inicio de sesión federado con Google (OAuth 2.0).



Pantalla de registro de nuevo usuario. El formulario captura las credenciales que Firebase Authentication utiliza para crear una cuenta mediante el método `createUserWithEmailAndPassword()`. Una vez registrado, Firebase asigna un UID único al usuario que se utilizará para asociar sus gastos en Cloud Firestore.

The image displays two side-by-side screenshots of a mobile application's registration screen. Both screenshots show a status bar at the top with the time 2:07, signal strength, and 56% battery. The app's header is a blue bar with the word "Registro" in white. Below the header, the text "Regístrate para comenzar a controlar tus gastos" is centered. The main content is a light purple rounded rectangle titled "Crear cuenta". It contains three input fields: "Correo electrónico", "Contraseña", and "Confirmar contraseña". A blue "Registrarse" button is positioned below the inputs, and a link "Ya tengo cuenta" is at the bottom. The left screenshot shows empty input fields, while the right screenshot shows the fields filled with "zkatya2629@gmail.com", ".....", and "....." respectively.

Registro

Regístrate para comenzar a controlar tus gastos

Crear cuenta

Correo electrónico

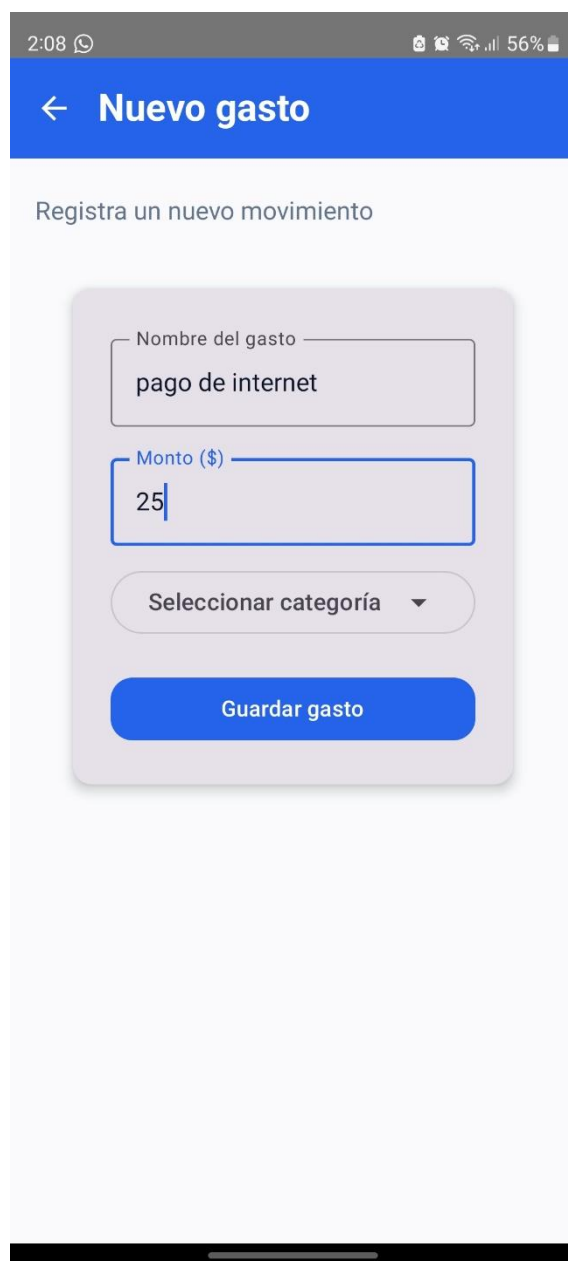
Contraseña

Confirmar contraseña

Registrarse

[Ya tengo cuenta](#)

Pantalla principal de la aplicación tras el primer inicio de sesión. Se muestra el total acumulado en \$0.00, indicando que no existen documentos registrados aún en la subcolección gastos del usuario en Cloud Firestore. El valor se calcula en tiempo real mediante una consulta a Firestore filtrada por el UID del usuario autenticado.



Pantalla de registro de nuevo gasto. El formulario recibe el nombre del gasto, el monto y la categoría. Al confirmar, los datos se persisten en Cloud Firestore bajo la ruta gastos/{userId}, asociando cada documento al UID del usuario autenticado para garantizar el aislamiento de datos entre usuarios.

2:08 56%

← Nuevo gasto

Registra un nuevo movimiento

Nombre del gasto
pago de internet

Monto (\$)
25

Servicios ▼

Guardar gasto

2:08 56%

← Nuevo gasto

Registra un nuevo movimiento

Comida

Transporte

Compras

Servicios

Entretenimiento

Salud

Educación

Otros

Nombre del gasto

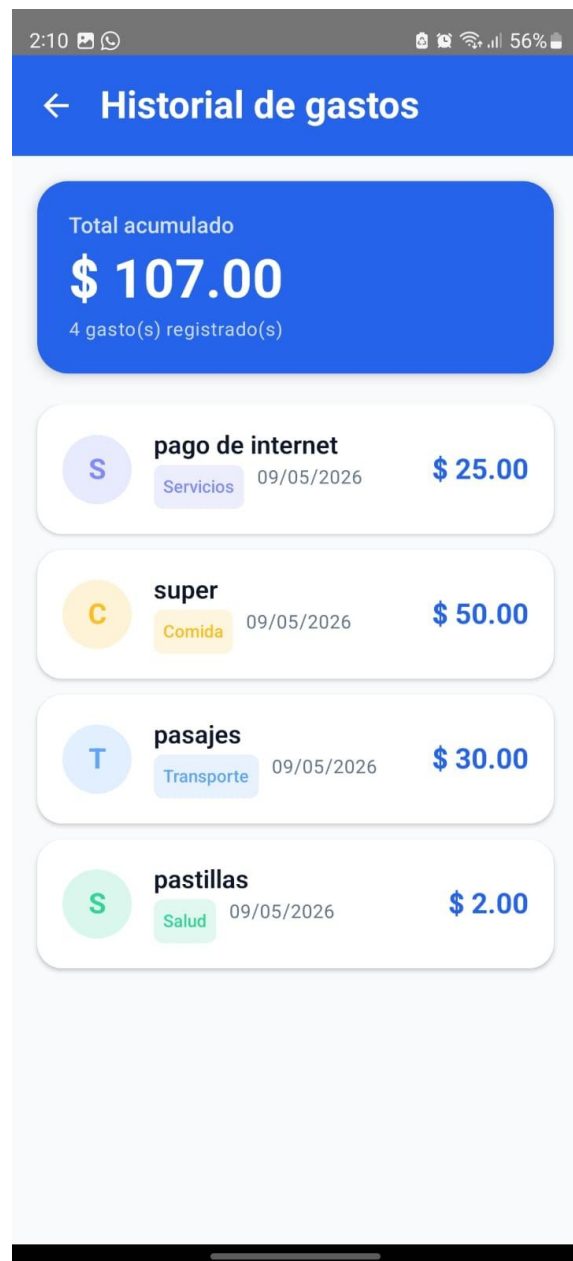
Monto (\$)

Servicios ▼

Guardar gasto

Actualización del total acumulado tras el registro de gastos. La aplicación consulta la colección de gastos en Firestore y suma los campos monto de todos los documentos asociados al usuario, mostrando el resultado en pantalla. El cálculo se realiza del lado del cliente una vez obtenida la lista de documentos.

Pantalla de historial de gastos. Se despliega la lista de documentos almacenados en Cloud Firestore para el usuario autenticado, ordenados por fecha de registro. Cada elemento muestra el nombre del gasto, la categoría, la fecha y el monto formateado con dos decimales.



Conclusiones

Las siguientes conclusiones reflejan nuestro análisis como equipo a partir de la investigación realizada y el proceso de desarrollo de la aplicación:

Esta investigación con respecto a Firebase Authentication me permitió comprender de una mejor manera la importancia de delegar la gestión de identidades a servicios especializados y que implementar un sistema de autenticación seguro desde cero implica un conocimiento profundo de criptografía, gestión de sesiones y protección frente a ataques que va más allá del alcance de la mayoría de los proyectos.

La implementación técnica de la autenticación con Firebase me demostró que la integración de servicios externos en Android con Kotlin es más accesible de lo que pensaba. La documentación oficial de Firebase es exhaustiva y los SDKs están bien diseñados. Mas sin embargo, el manejo de errores requiere de atención especial como: los diferentes códigos de error deben traducirse en mensajes entendibles para el usuario, lo que significa que debemos hacer un trabajo cuidadoso de diseño de la capa de presentación. Esta experiencia reforzó mi comprensión de la arquitectura cliente-servidor y el manejo asíncrono de operaciones en Kotlin a través de la práctica.

El desarrollo de la funcionalidad de gastos me permitió aplicar de manera práctica los conceptos de bases de datos NoSQL con Firestore. La estructura de colecciones y documentos requiere un diseño previo cuidadoso para optimizar las consultas. También aprendí que Firestore no admite consultas combinadas con múltiples filtros sin la creación de índices compuestos.

Mi responsabilidad en el diseño de la interfaz me hizo reflexionar sobre la importancia de la usabilidad en aplicaciones financieras. Una interfaz clara y organizada no es solo un requisito estético, sino funcional, pues el usuario debe de poder ingresar un gasto en pocos pasos y visualizar su resumen mensual de forma inmediata.

Conclusión general

En resumen, el desarrollo de esta aplicación de control de gastos constituyó a una experiencia de aprendizaje integral que va desde la investigación teórica sobre mecanismos de autenticación hasta la implementación práctica de una aplicación funcional con backend en la nube.

Como equipo identificamos principales aprendizajes, tales como: la importancia del diseño previo de la arquitectura de datos antes de escribir código, la necesidad de manejar apropiadamente los estados antes de escribir código, la necesidad de manejar apropiadamente los estados de error en operaciones asíncronas, y el valor de las buenas prácticas de seguridad como el uso de reglas de Firestore para proteger los datos de cada usuario.